



IIC1001 — Algoritmos y Sistemas Computacionales — 2023-1
Examen - Solución

Martes 4-Julio-2023

Duración: 150 minutos

Responder cada pregunta en una hoja separada.

1. [10p] En el curso de Introducción a la Programación hay N secciones, cada una con M estudiantes. Cada estudiante escribe un programa (código) con alguna cantidad de líneas.

- 1.1) [2p] Para dos programas, de largo L_1 y L_2 líneas respectivamente, se desea calcular un “índice de similitud”. Este valor se obtiene recorriendo cada línea del programa más largo y comparándola con cada una de las líneas del programa más corto. ¿Cuál es la complejidad asintótica de este algoritmo en notación $O()$?

R. Suponiendo que $L_1 \geq L_2$ (el otro caso es equivalente, o “sin pérdida de generalidad” como le gusta decir a los matemáticos), se deben recorrer las L_1 líneas, y por cada una de ellas hacer L_2 comparaciones. El total de comparaciones es $L_1 \times L_2$. Por lo tanto la complejidad asintótica es $O(L_1 L_2)$.

Puntaje:

2pts. Solución correcta. No es necesario detallar procedimiento.

1pto. Solución incorrecta, pero procedimiento con algún error menor, por ejemplo dejar constantes o términos de menor grado en la notación $O()$.

0pts. Solución incorrecta y no hay procedimiento o tiene errores importantes.

- 1.2) [2p] Se desea obtener el “índice de similitud” de todos los pares de programas posibles del curso. Suponiendo que todos los programas son del mismo largo L , ¿cuál es la complejidad asintótica de este algoritmo en notación $O()$?

R. Si todos los programas son de largo L , entonces comparar dos programas es $O(L^2)$. En total hay NM programas. Digamos que $P = NM$. En un conjunto de P elementos, la cantidad total de pares que se pueden formar es $(P(P-1))/2 = (1/2)P^2 - (1/2)P$, y esto es $O(P^2)$. Para cada uno de esos pares hay que hacer ejecutar un algoritmo que es $O(L^2)$. Por lo tanto la complejidad del procedimiento completo es $O(P^2 L^2)$. También puede expresarse como $O((NM)^2 L^2)$.

Puntaje:

2pts. Solución correcta. No es necesario detallar procedimiento.

1pto. Solución incorrecta, pero procedimiento con algún error menor, por ejemplo dejar constantes o términos de menor grado en la notación $O()$.

0pts. Solución incorrecta y no hay procedimiento o tiene errores importantes.

- 1.3) [3p] Supongamos que el “índice de similitud” se puede calcular en tiempo constante. Ahora queremos obtener los 10 pares de programas con mayor índice de similitud. Escoja su algoritmo de ordenamiento favorito (indicar el nombre, o describirlo si no recuerda el nombre), e indique la complejidad asintótica en notación $O()$ del algoritmo para obtener los 10 pares de programas con mayor índice de similitud.

R. Esta vez el cálculo del índice para un par de programas es $O(1)$. Para obtener los 10 pares con mayor índice de similitud hay que calcular el índice para cada par de programas. Hay $P = NM$ programas, y la cantidad de pares es $O(P^2)$. Luego hay que ordenar los $O(P^2)$ pares. Si usamos Quicksort o Mergesort ($O(n \log n)$), esto es $O(P^2 \log P^2)$. Finalmente hay quedarse con los 10 mayores elementos, lo cual se puede hacer en tiempo constante $O(1)$ ya que se trata de una cantidad fija. La complejidad que se obtiene es $O(P^2) + O(P^2 \log P^2) + O(1) = O(P^2 \log P^2)$.

Si utilizan algún algoritmo $O(n^2)$ como InsertionSort, SelectionSort, Bubblesort o alguno similar, la solución queda $O(P^2) + O(P^4) + O(1) = O(P^4)$.

No se puede utilizar algún algoritmo lineal como RadixSort o BucketSort porque la cantidad de programas no está acotada. Como dato, tampoco hablé de ellos en clase.

Puntaje:

3pts. Solución correcta. No es necesario detallar procedimiento.

2pto. Solución incorrecta, pero logra mencionar o describir el funcionamiento de un algoritmo de ordenamiento y determina su complejidad.

1pto. Solución incorrecta, pero logra mencionar o describir el funcionamiento de un algoritmo de ordenamiento, sin determinar su complejidad.

0pts. Solución incorrecta y no hay procedimiento o tiene errores importantes.

- 1.4) [3p] Supongamos que el “índice de similitud” se puede calcular en tiempo constante. Queremos ordenar las N secciones de acuerdo al valor del mayor “índice de similitud” encontrado en cada sección. ¿Cuál es la complejidad asintótica de este algoritmo en notación $O()$?

R. El cálculo del índice para un par de programas es $O(1)$. Encontrar el mayor índice en una sección es $O(M^2)$, ya que hay $O(M^2)$ pares de programas por sección. Ejecutar esto para las N secciones es $O(NM^2)$. Una vez que se tienen los N índices (uno por cada sección), hay que ordenarlos y eso se puede hacer en $O(N^2)$ ó $O(N \log N)$. La complejidad del procedimiento queda en $O(NM^2) + O(N^2)$, o bien $O(NM^2) + O(N \log N)$ dependiendo del algoritmo de ordenamiento.

Puntaje:

3pts. Solución correcta. No es necesario detallar procedimiento.

2pto. Solución incorrecta, pero procedimiento con algún error menor, por ejemplo dejar constantes o términos de menor grado en la notación $O()$.

1pto. Solución incorrecta y procedimiento con varios errores.

0pts. Solución incorrecta y no hay procedimiento o tiene errores importantes.

No es necesario explicitar el desarrollo, pero puede ayudar a obtener puntaje parcial.

2. [10p] Se tiene un archivo con N postulaciones de estudiantes a 3 universidades en orden de preferencia.

2.1) [5p] Se desea obtener una lista con todas las instituciones que aparecen en el archivo de postulaciones, **sin repetirlas**. ¿Qué estructura de datos vista en este curso usaría para obtener esa lista de manera más eficiente? Indique cuál sería el algoritmo y su complejidad.

R. Entre las estructuras vistas en el curso, la más eficiente es la tabla de *hash*. El algoritmo consiste en aplicar la función de *hash* a cada elemento de la lista e insertarlo en la tabla. Si no se produce colisión, entonces se agrega el elemento a una lista resultado. En cambio si, al insertar el elemento, se produce una colisión, entonces ese elemento está duplicado y no se debe agregar a la lista resultado. Si hay n universidades en la lista, el proceso implica recorrer una vez cada uno de los n elementos de la lista y aplicar el hash, que debería ser $O(1)$. La complejidad resultante es $O(n)$.

Con alguna estructura secuencial, como lista (*array* o *linked-list*), cola o *stack*, es necesario comparar cada elemento de nuevo con los ya insertados en la lista. Eso hace que la complejidad llegue a $O(n^2)$ en el peor caso.

Otro algoritmo puede ser ordenar la lista (mejor caso $O(n \log n)$, y luego recorrerla eliminando duplicados (que estarían consecutivos). Eso es $O(n \log n + n) = O(n \log n)$, que es menos eficiente que lineal.

Puntaje:

5 pts. Estructura *hash*, descripción de algoritmo que lo usa, y complejidad lineal.

4 pts. Estructura *hash*, descripción de algoritmo o de complejidad con algún error menor.

3 pts. Estructura *hash*, falta descripción de algoritmo, o complejidad, o hay un error importante en alguna.

2 pts. Otra estructura, pero con descripción de algoritmo y complejidad correctos y consistentes.

1 pts. Otra estructura. Descripción de algoritmo o complejidad con errores o no existentes.

0 ptos. Otra estructura, sin algoritmo ni complejidad.

2.2) [5p] Supongamos que tenemos una lista donde cada elemento es un par (u_i, p_i) que indica una universidad y la cantidad de personas que postula a ella. Por ejemplo: $\{(u_0, p_0), (u_1, p_1), (u_2, p_2), (u_3, p_3), \dots\}$. La estructura está ordenada de mayor a menor de acuerdo a los valores p_i . Queremos insertar un nuevo elemento (u_k, p_k) manteniendo el orden. Podemos implementar la lista como un arreglo (*array*), una lista ligada (*linked list*) o un *stack*. ¿Qué implementación permite efectuar la operación de manera más eficiente y por qué?

R. Al usar un *stack* hay que sacar $n - k$ elementos antes de ingresar el nuevo. En el peor caso habría que sacarlos todos. Al usar una *array* se puede encontrar el elemento rápidamente, pero al insertarlo hay que mover los siguientes $n - k$ elementos que le siguen para mantener la propiedad de que todos los elementos se encuentran contiguos. Si se usa una *linked-list* el costo mayor es encontrar la posición, pero una vez encontrada no hay que hacer movimientos pues solo se modifican los punteros del elemento anterior. La estructura más eficiente para esto es la *linked-list*,

Puntaje:

5 pts. Estructura *linked-list*, y explicación correcta.

4 pts. Estructura *linked-list*, y explicación con errores menores.

3 pts. Estructura *linked-list*, y explicación con errores importantes o sin explicación.

2 pts. Estructura *array*, y explicación consistente.

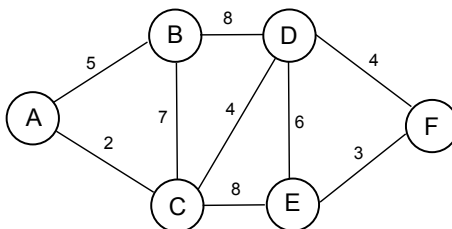
1 pts. Estructura *array*, sin explicación o explicación con errores importantes.

0 pts. Otra estructura, independiente de la explicación.

3. [10p] Se define el grafo no dirigido $G = (V, E)$, donde $V = \{A, B, C, D, E, F\}$, y el conjunto E está dado por $E = \{(A, B, 5), (A, C, 2), (B, C, 7), (B, D, 8), (C, D, 4), (C, E, 8), (D, E, 6), (D, F, 4), (E, F, 3)\}$. Cada elemento (X, Y, W) indica que la arista (X, Y) tiene costo W . El nodo inicial S será el último dígito de su número de alumno de acuerdo al *hash*: $\{(0, 6) \rightarrow A, (1, 7) \rightarrow B, (2, 8) \rightarrow C, (3, 9) \rightarrow D, 4 \rightarrow E, J \rightarrow F\}$

3.1) [2p] Dibuje el grafo de acuerdo a la descripción, e indique su número de alumno y nodo S .

R.



Puntaje:

2 pts. Correcto.

1 pts. 1 o 2 errores.

0 pts. Más de 2 errores.

3.2) [4p] Realice un recorrido DFS a partir del nodo S .

R. Múltiples respuestas posibles, dependiendo del nodo S y el orden en que evalúan los vecinos. Para estos casos se supondrá orden alfabético, pero podrían tomar otro.

- $\text{DFS}(A) = \{A, B, C, D, E, F\}$
- $\text{DFS}(B) = \{B, A, C, D, E, F\}$
- $\text{DFS}(C) = \{C, A, B, D, E, F\}$
- $\text{DFS}(D) = \{D, B, A, C, E, F\}$
- $\text{DFS}(E) = \{E, C, A, B, D, F\}$
- $\text{DFS}(F) = \{F, E, C, A, B, D\}$

Puntaje: No castigar errores de arrastre por error al dibujar el grafo. Resultado debe ser consistente.

4 pts. Resultado posible de acuerdo al nodo S .

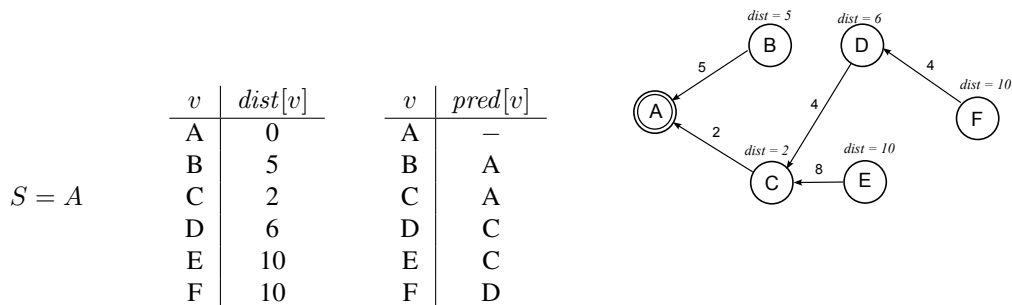
3 pts. Error menor. Por ejemplo, no incluir nodo S como primer nodo.

2 pts. Error importante.

0 pts. Incorrecto

3.3) [4p] Utilice el algoritmo de Dijkstra para obtener las rutas más cortas desde S hasta los demás nodos. Debe mostrar el camino desde S hacia cada uno de los otros nodos, y el costo de la ruta. No es necesario mostrar el desarrollo, pero puede permitir obtener puntaje parcial.

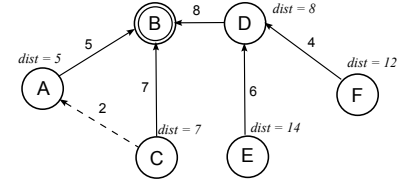
R. Múltiples respuestas posibles, dependiendo del nodo S .



$S = B$

v	$dist[v]$
A	5
B	0
C	7
D	8
E	14
F	12

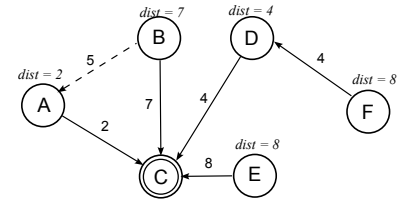
v	$pred[v]$
A	B
B	—
C	B (ó A)
D	B
E	D
F	D



$S = C$

v	$dist[v]$
A	2
B	7
C	0
D	4
E	8
F	8

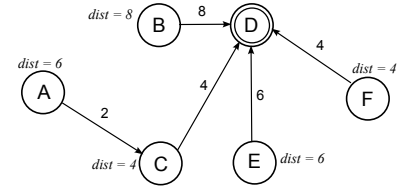
v	$pred[v]$
A	C
B	C (ó A)
C	—
D	C
E	C
F	D



$S = D$

v	$dist[v]$
A	6
B	8
C	4
D	0
E	6
F	4

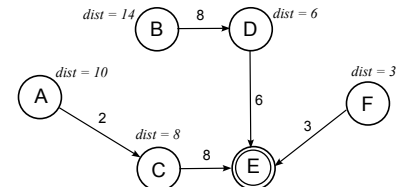
v	$pred[v]$
A	C
B	D
C	D
D	—
E	D
F	D



$S = E$

v	$dist[v]$
A	10
B	14
C	8
D	6
E	0
F	3

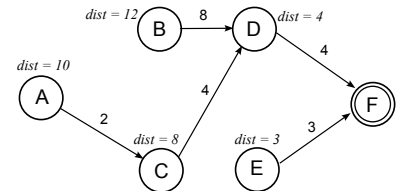
v	$pred[v]$
A	C
B	D
C	E
D	E
E	—
F	E



$S = F$

v	$dist[v]$
A	10
B	12
C	8
D	4
E	3
F	0

v	$pred[v]$
A	C
B	D
C	D
D	F
E	F
F	—



Es suficiente con mostrar ambas tablas ($dist$ y $pred$), o bien el grafo dibujado si es que contiene la misma información. Algunos casos podían tener rutas alternativas que igualmente son de menor costo (punteadas en los dibujos).

No castigar errores de arrastre por error al dibujar el grafo. Resultado debe ser consistente.

Puntaje:

4 pts: Información de *dist* y *pred* correcta.

3 pts: Un error en *dist* o en *pred*.

2 pts: Un error en *dist* y un error en *pred*. O bien está solo una las tablas.

1 pts: Más de un error en *dist* y en *pred*

0 pts: Más errores que en el caso anterior.

4. [10p] Construya un archivo, utilizando Bytes (en notación hexadecimal) de acuerdo al siguiente formato:

- [1p] 8 Byte, donde cada byte contiene el **caracter** ASCII correspondiente a la fecha de hoy en formato YYYYMMDD. Atención que son dos caracteres para el mes y dos para el día.

R. La fecha actual es 20230704. Se solicita escribir los byte correspondientes a cada **caracter ASCII**, por lo tanto hay que escribir 8 Byte: el Byte que representa el caracter 2, el Byte que representa el caracter 0, etc.

Caracter ASCII	2	0	2	3	0	7	0	4
Byte	32	30	32	33	30	37	30	34

Puntaje:

1 pto: Correcto

0 pto: Resultado incorrecto, o bien no utiliza caracteres ASCII, o no usa los 8 Byte.

- [1p] 1 Byte con el valor ASCII correspondiente a E.

Caracter ASCII	E
Byte	45

Puntaje:

1 pto: Correcto

0 pts: Incorrecto

- [1p] 1 Byte con el valor numérico 12 si usted está en la sala E12, o el valor numérico 13 si está en la E13.

R. Ahora se pide el **valor numérico** 12 ó 13. Cada uno de estos valores cabe en un Byte, pues un Byte permite almacenar hasta el valor 255. Si se pidieran los caracter ASCII, necesitaríamos 3 Byte (uno por cada caracter), pero no es el caso.

Valor	12 ó 13
Byte	0C ó 0D

Puntaje:

1 pto. Correcto o error muy menor.

0 pto. Incorrecto. Por ejemplo, ocupar más de 1 byte. No anteponer el 0.

- [2p] 1 Byte con el valor numérico que representa la cantidad de Byte usados en la siguiente parte.

R. El valor numérico *X* seguramente será 3 o 4 Byte (siguiente paso), por lo tanto el Byte que se debe agregar aquí es 03 ó bien 04.

Puntaje:

2 pts. Correcto.

1 pto. Error muy menor. No anteponer el 0.

0 pto. Incorrecto. Por ejemplo, ocupar más de 1 byte.

- [4p] Tome su número de alumno y separe los dígitos consecutivos en grupos de manera que queden números entre 0 y 255. Considere la J como un 0. Luego escriba cada número como Byte. Por ejemplo, mi número de alumno es 97624632. Al separarlo en grupos queda 97 62 46 32. Cada uno de esos cuatro valores debe ser escrito como Byte. Si el número de alumno fuese 2163890J, entonces la separación sería 216 38 90 0, y esos valores deberían ser escritos como 4 Byte.

R. El número de alumno debe separarse en grupos de dígitos de manera que siempre quedan valores menores a 255, y luego convertir cada uno a Byte. A continuación 2 ejemplos:

Decimal	97	62	46	32
Byte	61	3E	2E	20
Decimal	216	38	90	0
Byte	D8	26	5A	00

Puntaje

4 pts. Correcto.

3 pts. Un error. Ejemplos: no están bien agrupados los dígitos, pero el resto del procedimiento está OK

2 pto. Dos o tres errores distintos.

0 pts. Más de 3 errores. Incorrecto.

- **[1p]** El archivo debe quedar con 16 bytes en cada línea. Si es necesario debe agregar el byte correspondiente al valor numérico 255 al final, y repetirlo hasta completar una línea de 16 byte.

El valor numérico 255 en Byte corresponde a FF. Si escribimos los Byte usando 16 Byte por línea, queda:

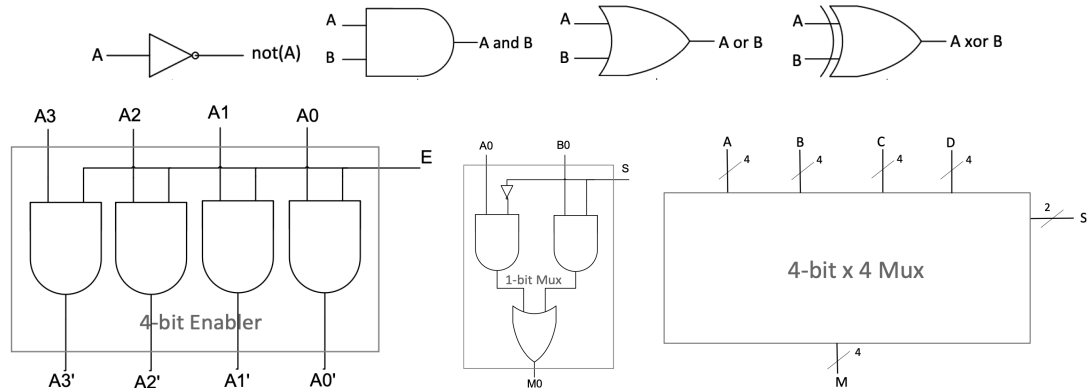
Posición	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Byte	32	30	32	33	30	37	30	34	45	0C	04	61	3E	2E	20	FF

Puntaje:

1 pto. Correcto o error muy menor.

0 pto. Incorrecto.

5. [10p] Tomando como referencia los siguientes componentes, indique la salida correspondiente para cada entrada indicada. Debe indicar cada uno de los bits solicitados. Como referencia se agregan los diagramas vistos en clases.



- 5.1) [1p] 4-bit Enabler. Entrada: $A = A_3A_2A_1A_0 = 0xC$, $E = 1$
R. Si $A = A_3A_2A_1A_0 = 0xC = 1100$, entonces $A' = A'_3A'_2A'_1A'_0 = 1100 = 0xC$
- 5.2) [1p] 4-bit Enabler. Entrada: $A = A_3A_2A_1A_0 = 0xC$, $E = 0$
R. Si $A = A_3A_2A_1A_0 = 0xC = 1100$, entonces $A' = A'_3A'_2A'_1A'_0 = 0000 = 0x0$
- 5.3) [2p] 1-bit Mux. Entrada: $A_0 = 1$, $B_0 = 0$, $S = 0$
R. $M_0 = A_0 = 1$
- 5.4) [2p] 1-bit Mux. Entrada: $A_0 = 1$, $B_0 = 0$, $S = 1$
R. $M_0 = B_0 = 0$
- 5.5) [2p] 4-bit x 4 Mux. Ent0: $A = 0xC$, Ent1: $B = 0xA$, Ent2: $C = 0x6$, Ent3: $D = 0x3$ $S = S_1S_0 = 01$
R. Si $S = S_1S_0 = 01$, entonces pasa la Entrada 1, y queda $M = 0xA = M_3M_2M_1M_0 = 1010$.
- 5.6) [2p] 4-bit x 4 Mux. Ent0: $A = 0xC$, Ent1: $B = 0xA$, Ent2: $C = 0x6$, Ent3: $D = 0x3$ $S = S_1S_0 = 10$
R. Si $S = S_1S_0 = 10$, entonces pasa la Entrada 2, y queda $M = 0x6 = M_3M_2M_1M_0 = 0110$.

6. [10p] Responda de manera breve y precisa las siguientes preguntas:

6.1) [2p] ¿Cuál es el rol de la ALU (unidad aritmético-lógica) dentro de un computador?

R. La ALU se encarga de ejecutar operaciones aritméticas o lógicas de acuerdo a los valores que recibe de *input* desde los registros, y los valores de control que recibe.

Puntaje:

2 pts. Correcto. Menciona ejecución de operaciones, y existencia de valores de control

1 pto. Solo menciona operaciones, o solo menciona elementos de control.

0 pts. Ninguno de los dos elementos (operaciones o control) presentes.

6.2) [2p] ¿Cuál es el rol del PC (Program Counter) en un computador?

R. El rol del PC es indicar la posición de memoria en que se encuentra la siguiente instrucción a ejecutar del código (programa, ó proceso). Menos formalmente se puede decir que indica “en qué parte va” el programa.

Puntaje:

2 pts. Correcto. Menciona que es una indicación (registro) dentro del código del programa (puede decir proceso, código binario, pero es un error menor decir línea o código fuente). También indica que lo que se lee una instrucción.

1 pto. Hable de “en qué parte va” o de que es un registro; o bien habla de las instrucciones, pero no habla de ambas cosas.

0 pts. Incorrecto. Ninguno de los elementos importantes presentes.

6.3) [2p] El planificador (*scheduler*) del sistema operativo permite decidir qué proceso se ejecuta a continuación. Dos tipos de *scheduler* son el First-Come First-Served (FCFS) que elige por orden de llegada; o bien el Round-Robin, que otorga un intervalo (*quantum*) de tiempo a cada proceso para ejecutar antes de devolverlo a la cola. ¿Cuál de esos *schedulers* es más apropiado para un sistema moderno y por qué?

R. Round-robin es más apropiado. Evita que los procesos esperen a que otro termine. También se puede decir que reduce el tiempo de espera entre procesos, o hacer referencia al tamaño de los cuántum de cada proceso.

Puntaje:

2 pts. Algoritmo Round-Robin. Hace referencia al tiempo de espera, o a la interactividad, o a no tener que esperar que otro proceso termine.

1 pto. Algoritmo correcto y justificación inexistente o errónea. O bien, algoritmo incorrecto pero explicación consistente.

0 pts. Algoritmo incorrecto y sin explicación, o explicación con errores importantes.

6.4) [2p] El sistema operativo evita que un proceso pueda acceder a memoria de otro proceso. ¿Por qué esto es necesario?

R. Es importante porque cada proceso puede manejar datos privados, o porque un proceso podría leer o escribir memoria de otro proceso afectando su funcionamiento.

Puntaje:

2 pts. Una razón correcta. Puede hacer referencia a protección, a no-interferencia de procesos, o temas de seguridad.

1 pto. Error menor, o razón incompleta.

0 pts. Erro importante.

6.5) [2p] Un problema al asignar memoria a un proceso para que pueda ejecutar, es que puede que haya memoria disponible, pero no de manera contigua. ¿Cómo solucionan esto los sistemas operativos modernos?

R. Dos elementos: asignan memoria en bloques del mismo tamaño (páginas), y hay una manera (tabla de páginas, función, puntero) de que cada proceso crea que tiene memoria continua usando direcciones virtuales en lugar de direcciones físicas y un mecanismo de conversión.

Puntaje:

2 pts. Referencia dos aspectos. (1) el tamaño de los bloques (puede hablar o no de páginas o *frames* o marcos). (2) Habla de un mecanismo de transformación, que puede ser tabla de páginas, alguna función, o bien memoria virtual.

1 pto. Uno de los aspectos presente, pero el otro no.

0 pts. Ningún aspecto presente.